

循序渐进，学习开发 xv6-riscv

第 1 章 建立开发环境

汪辰



目录

01

构建系统

02

工具链

03

运行与调试



01

构建系统

构建系统

A build system or build automation tool is a tool or set of tools that automate the compilation and linking of source code into an executable program or library. They streamline the software development process by managing dependencies, resolving conflicts, and ensuring consistent builds across different environments.

摘录自维基百科

构建系统 - Make



- make <https://www.gnu.org/software/make/>

构建系统 - Make



- make <https://www.gnu.org/software/make/>

```
$ sudo apt install build-essential
```

```
$ make -v
```

```
GNU Make 4.3
```

```
Built for x86_64-pc-linux-gnu
```

```
Copyright (C) 1988-2020 Free Software Foundation, Inc.
```

```
License GPLv3+: GNU GPL version 3 or later
```

```
<http://gnu.org/licenses/gpl.html>
```

```
This is free software: you are free to change and redistribute it.
```

```
There is NO WARRANTY, to the extent permitted by law.
```

构建系统 - Make



- make <https://www.gnu.org/software/make/>
- Makefile: 配合 make，用于描述构建工程过程中所管理的对象以及如何构造工程的过程。由一条或者多条 **规则 (rule)** 组成，每条规则由三要素构成：
 - target: 目标，可以是 obj 文件也可以是可执行文件
 - prerequisites: 生成 target 所需要的依赖
 - command: 为了生成 target 需要执行的命令，可以有多条

必须是一个 TAB

```
target...:prerequisites...  
    command...  
    ...
```

```
hello: hello.c  
    gcc hello.c -o hello
```

构建系统 - Make

Makefile 中其他元素简介:

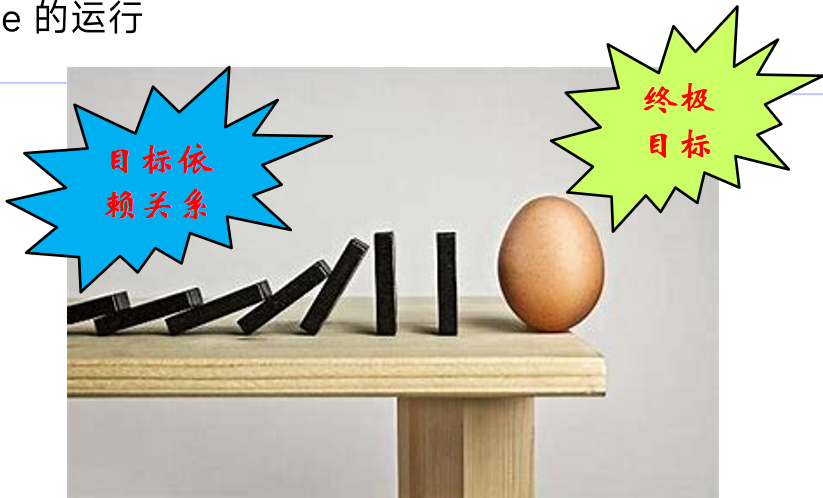
- 缺省规则（不指明默认第一个）
- 伪规则

```
clean:
    rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
        */*.o */*.d */*.asm */*.sym \
        $K/kernel fs.img \
        mkfs/mkfs .gdbinit \
        $U/usys.S \
        $(UPROGS)
```

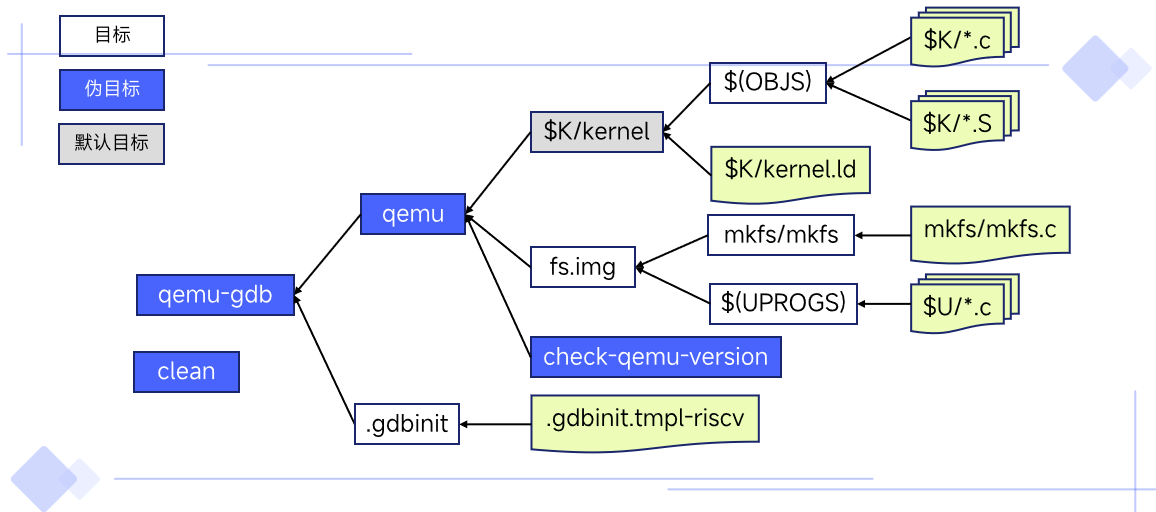
- 注释: 行注释, 以“#”开头

构建系统 - Make

make 的运行



构建系统 - Makefile 分析





02

工具链



工具链

A toolchain is a set of software development tools used to build and otherwise develop software. Often, the tools are executed sequentially and form a pipeline such that the output of one tool is the input for the next.

A cross compiler is a compiler capable of creating executable code for a platform other than the one on which the compiler is running.

摘录自维基百科

工具链 - GCC

GCC (GNU Compiler Collection) <https://gcc.gnu.org/>



- 由 GNU 开发的，遵循 GPL 许可证发行的编译器套件。
- 支持 C、C++、Objective-C、Fortran、Ada 和 Go 语言等多种语言前端，已被移植到多种计算机体系架构上，如 x86、ARM、RISC-V 等。
- GCC 的初衷是为 GNU 操作系统专门编写一款编译器，现已被大多数“Unix-like”操作系统（如 Linux、BSD、MacOS 等）采纳为标准的编译器。

工具链 - GCC

GCC (GNU Compiler Collection) <https://gcc.gnu.org/>



- 由 GNU 开发的可移植、许可证发行的编译器和库

- 支持 \$ sudo apt install gcc-riscv64-unknown-elf

言等

如 x8

```
$ riscv64-unknown-elf-gcc --version  
riscv64-unknown-elf-gcc (13.2.0-11ubuntu1+12) 13.2.0  
Copyright (C) 2023 Free Software Foundation, Inc.
```

- GCC This is free software; see the source for copying conditions. There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

现已

MacC

工具链 - xv6 的编译选项

在 xv6 的 Makefile 中 gcc 的编译选项定义在 CFLAGS 中。部分重要选项：

选项	含义
-Wall	使能所有告警
-Werror	所有告警（warning）都认为是错误（error），错误会导致编译退出，而告警不会。
-Wno-unknown-attributes	忽略所有不认识的属性，不报错。避免不同编译器或者同一个编译器的不同版本对 C 语言处理的差异的不同导致的警告。
-O	对编译输出大小和执行时间进行最基本的优化，等同于 -O1，是推荐的优化级别。
-ggdb	和 -g 类似，但针对 GDB 优化，适用于确定只用 GDB 调试的场景。
-gdwarf-2	指定调式信息格式为 DWARF（版本 2）。
-march=rv64gc	rv64 指定目标架构为 RISC-V 64位; g 包含通用整数指令集（IMAFD）；c 指定压缩指令集扩展。
-mcmmodel=medany	定义内存寻址的方式为 medany（Medium Any），即程序的最终内存布局在一个 4GB 的窗口内（以 PC 为中心的 正负 2GB 范围内）从而可以生成高效的 PC 相对寻址指令进行访存
.....	

工具链 - LD

LD (GNU Linker) <https://sourceware.org/binutils/>

- 由 GNU 开发的，遵循 GPL 许可证发行的 binutils 套件中的一个程序。
- GNU 链接器 LD 是一个将编译后的目标文件和库组合成可执行程序或共享库的核心工具。

工具链 - LD

LD (GNU Linker) <https://sourceware.org/binutils/>

- 由 GNU 开发并由其许可证发行的 binutils 套

件中

```
$ sudo apt install gcc-riscv64-unknown-elf
```

- GNU

成可

```
$ riscv64-unknown-elf-ld --version
```

```
GNU ld (2.42-1ubuntu1+6) 2.42
```

```
Copyright (C) 2024 Free Software Foundation, Inc.
```

```
This program is free software; you may redistribute it under the terms of  
the GNU General Public License version 3 or (at your option) a later version.
```

```
This program has absolutely no warranty.
```

工具链 - xv6 的链接选项

GCC 控制 ld 行为的方式:

- 最通用的方法是使用 `-Wl,` 选项, 告诉 GCC 驱动程序, 将紧随其后的选项原封不动地传递给链接器 ld: 例子: `gcc -Wl,-Rpwd,-rpath,/usr/lib main.o -o main`
- 一些专门用来控制链接过程的专用选项, 被 GCC 解释后传递给 ld, 譬如 `-lxxx`, `-Lxxx`, `-shared`, `-static` 等

在 xv6 的 Makefile 中 gcc 的链接选项定义在 LDFLAGS 中。

选项	含义
<code>-z max-page-size=4096</code>	指定链接器将最终生成的程序中的各个 section 的起始地址都是 max-page-size 值的整数倍。

工具链 - xv6 的链接脚本

链接脚本类似一个“说明书”，它告诉链接器如何将输入的目标文件（.o 文件）和库文件（.a 文件）中的各个“段（section）”组合起来，生成最终的可执行文件或库文件。通过链接脚本我们可以理解和掌握程序底层内存布局。

- GNU ld 使用的 Linker Script 是简单的纯文本文件，采用特定的脚本描述语言编写。
- 每个 Linker Script 中包含有多条命令（Command）
- 注释采用 “/*” 和 “*/” 括起来
- 在使用 GCC 工具链时，通过 -T 选项指定链接脚本

```
$K/kernel: $(OBJS) $K/kernel.ld
$(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
$(OBJDUMP) -S $K/kernel > $K/kernel.asm
$(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $K/kernel.sym
```

```
_%: %.o $(ULIB) $U/user.ld
$(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
$(OBJDUMP) -S $@ > $*.asm
$(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym
```

工具链 - xv6 的链接脚本 kernel/kernel.ld

```
OUTPUT_ARCH( "riscv" )
ENTRY( _entry )
SECTIONS
{
    . = 0x80000000;
    .text : {
        kernel/entry.o(_entry)
        *(.text .text.*)
        . = ALIGN(0x1000);
        _trampoline = .;
        *(trampsec)
        . = ALIGN(0x1000);
        ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
        PROVIDE(etext = .);
    }
    .rodata : {
        // .....
    }
    .data : {
        // .....
    }
    .bss : {
        // .....
    }
    PROVIDE(end = .);
}
```

工具链 - xv6 的链接脚本 kernel/kernel.ld

OUTPUT_ARCH("riscv")

指定目标架构为 RISC-V

```
ENTRY( _entry )
SECTIONS
{
    . = 0x80000000;
    .text : {
        kernel/entry.o(_entry)
        *(.text .text.*)
        . = ALIGN(0x1000);
        _trampoline = .;
        *(trampsec)
        . = ALIGN(0x1000);
        ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
        PROVIDE(etext = .);
    }
    .rodata : {
        // .....
    }
    .data : {
        // .....
    }
    .bss : {
        // .....
    }
    PROVIDE(end = .);
}
```

工具链 - xv6 的链接脚本 kernel/kernel.ld

```
OUTPUT_ARCH( "riscv" )
```

```
ENTRY( _entry )
```

```
SECTIONS
```

```
{
```

```
  . = 0x80000000;
```

```
  .text : {
```

```
    kernel/entry.o(_entry)
```

```
    *(.text .text.*)
```

```
    . = ALIGN(0x1000);
```

```
    _trampoline = .;
```

```
    *(trampsec)
```

```
    . = ALIGN(0x1000);
```

```
    ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
```

```
    PROVIDE(etext = .);
```

```
}
```

```
  .rodata : {
```

```
    // .....
```

```
}
```

```
  .data : {
```

```
    // .....
```

```
}
```

```
  .bss : {
```

```
    // .....
```

```
}
```

```
  PROVIDE(end = .);
```

```
}
```

设置程序的入口点为 `_entry` 符号
(定义在 `kernel/entry.S`)
`_entry` 符号对应的地址会写入 ELF
头部的 `e_entry` 字段。

工具链 - xv6 的链接脚本 kernel/kernel.ld

```
OUTPUT_ARCH( "riscv" )
ENTRY( _entry )
SECTIONS
{
  . = 0x80000000;
  .text : {
    kernel/entry.o(_entry)
    *(.text .text.*)
    . = ALIGN(0x1000);
    _trampoline = .;
    *(trampoline)
    . = ALIGN(0x1000);
    ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
    PROVIDE(etext = .);
  }
  .rodata : {
    // .....
  }
  .data : {
    // .....
  }
  .bss : {
    // .....
  }
  PROVIDE(end = .);
}
```

设置内核的启动地址是 0x80000000

工具链 - xv6 的链接脚本 kernel/kernel.ld

```
OUTPUT_ARCH( "riscv" )
ENTRY( _entry )
SECTIONS
{
    . = 0x80000000;
    .text : {
        kernel/entry.o(_entry),
        *(.text .text.*)
        . = ALIGN(0x1000);
        _trampoline = .;
        *(trampsec)
        . = ALIGN(0x1000);
        ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
        PROVIDE(etext = .);
    }
    .rodata : {
        // .....
    }
    .data : {
        // .....
    }
    .bss : {
        // .....
    }
    PROVIDE(end = .);
}
```

代码指令

只读数据

数据（可读写）

未初始化数据（可读写）

工具链 - xv6 的链接脚本 kernel/kernel.ld

```
OUTPUT_ARCH( "riscv" )
```

```
ENTRY( _entry )
```

```
SECTIONS
```

```
{
```

```
  . = 0x80000000;
```

```
  .text : {
```

```
    kernel/entry.o(_entry)
```

```
    *(.text .text.*)
```

```
    . = ALIGN(0x1000);
```

```
    _trampoline = .;
```

```
    *(trampsec)
```

```
    . = ALIGN(0x1000);
```

```
    ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
```

```
    PROVIDE(etext = .);
```

```
}
```

```
  .rodata : {
```

```
    // .....
```

```
}
```

```
  .data : {
```

```
    // .....
```

```
}
```

```
  .bss : {
```

```
    // .....
```

```
}
```

```
  PROVIDE(end = .);
```

```
}
```

确保 entry.o 中的 _entry 段放在最前面

所有参与链接的目标文件中的 .text/.text.* section 集中排放在这里

当前地址对齐到 4KB 边界

定义符号 _trampoline, 标记 trampoline 开始地址

trampsec section 排放在这里 (定义在 kernel/trampoline.S)

当前地址再次对齐到 4KB

确保 trampoline 正好占用一个页面 (4KB)

定义 etext 符号, 标记代码段结束

工具链 - xv6 的链接脚本 kernel/kernel.ld

```
OUTPUT_ARCH( "riscv" )
ENTRY( _entry )
SECTIONS
{
    . = 0x80000000;
    .text : {
        kernel/entry.o(_entry)
        *(.text .text.*)
        . = ALIGN(0x1000);
        _trampoline = .;
        *(trampsec)
        . = ALIGN(0x1000);
        ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
        PROVIDE(etext = .);
    }
    .rodata : {
        // .....
    }
    .data : {
        // .....
    }
    .bss : {
        // .....
    }
    PROVIDE(end = .);
}
```

定义结束标记符号 end

更多参考

循序渐进，学习开发一个
RISC-V 上的操作系统



扫码听课

从零开始为 RISC-V
构建一个 Linux 系统



扫码听课



03

运行与调试



<https://www.qemu.org/>



QEMU (Quick EMUlator) 是一套由 (Fabrice Bellard) 编写的以 GPL 许可证分发源码的计算机系统模拟软件，在 GNU/Linux 平台上使用广泛。

- 支持多种体系架构。譬如：IA-32 (x86), AMD 64, MIPS 32/64, RISC-V 32/64 等等。
- QEMU 有两种主要运作模式：
 - ✓ User mode：直接运行应用程序。
 - ✓ System mode：模拟整个计算机系统，包括中央处理器及其他周边设备。

运行



<https://www.qemu.org/>

QEMU (Quick Emulator) 是一套由 (Fabrice Bellard) 编写的以 GPL 协议发布的虚拟机软件。

GNU/Linux 上安装 QEMU 的命令如下：

```
$ sudo apt install qemu-system-misc
```

- 支持 RISC-V 架构的 QEMU 安装和运行。

```
$ qemu-system-riscv64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.13)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project developers
```

- QEMU 有两种主要运行模式。

- ✓ User mode：直接运行应用程序。

- ✓ System mode：模拟整个计算机系统，包括中央处理器及其他周边设备。

运行

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

命令选项	含义
-machine virt	指定要模拟的虚拟机的机器类型为 virt
-m 128M	设置虚拟机的物理内存（RAM）大小为 128M
-smp 3	指定模拟器模拟的 CPU 系统架构类型为 SMP，模拟 3 个 HART
-bios none	告诉 QEMU 不要自动加载或运行任何默认的固件（对 riscv 是 openSBI）
-kernel kernel/kernel	指定加载操作系统内核二进制文件所在位置
-nographic	禁用所有图形输出，将模拟的串口设备连接到宿主机的标准输入（stdin，即宿主机的键盘）和标准输出（stdout，即宿主机上运行的控制台终端）
.....	

运行

```
$ make qemu  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -  
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0  
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

xv6 kernel is booting

hart 2 starting

hart 1 starting

init: starting sh

\$ echo hello

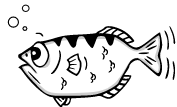
hello

\$ QEMU: Terminated

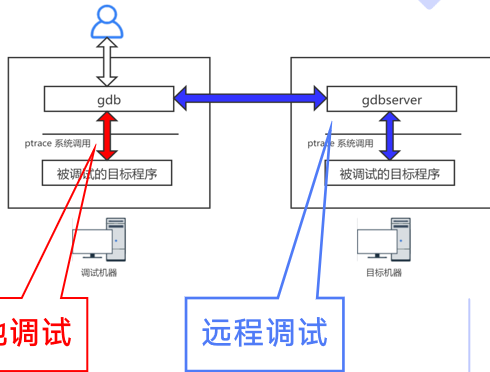
强制退出 QEMU，按 Ctrl-a, x（先同时按下 Ctrl 和 a，同时松开再按下 x）

调试

<https://www.gnu.org/software/gdb/>

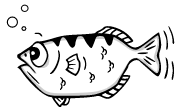


- GDB (GDB: The GNU Project Debugger), GNU 项目调试器, 用于查看另一个程序在执行过程中正在执行的操作, 或该程序崩溃时正在执行的操作。
- 被调试的程序可能与 GDB 在同一台计算机上执行, 也可能在另一台计算机 (远程) 上或者在模拟器上执行。
- GDB 支持调试多种语言: 譬如: Assembly, C, Go, Rust,



调试

<https://www.gnu.org/software/gdb/>



- GDB (GDB: The GNU Project Debugger), 调试器, 用于查看正在执行的程序的操作



- 被调试的计算机 (远程支持)
 - \$ sudo apt install gdb-multiarch
 - \$ gdb-multiarch --version
 - GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
 - Copyright (C) 2024 Free Software Foundation, Inc.
 - License GPLv3+: GNU GPL version 3 or later
 - <<http://gnu.org/licenses/gpl.html>>
 - This is free software: you are free to change and redistribute it.
 - There is NO WARRANTY, to the extent permitted by law.
- GDB 支持 Assembly, C, Go, Rust,

调试

```
$ make qemu-gdb  
sed "s/:1234/:26000/" < .gdbinit.tmpl-riscv > .gdbinit  
*** Now run 'gdb' in another window.  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -  
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0  
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

命令选项	含义
-S	让虚拟机在第一条指令之前就停下来，等待调试器（GDB）的后继指令。
-gdb tcp::26000	通过指定 TCP 端口告诉 QEMU 在该端口上开启 GDB 调试服务器

调试

```
$ CPUS=1 make qemu-gdb  
sed "s/:1234/:26000/" < .gdbinit.tmpl-riscv > .gdbinit  
*** Now run 'gdb' in another window.  
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -  
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0  
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

命令选项	含义
-S	让虚拟机在第一条指令之前就停下来，等待调试器（GDB）的后继指令。
-gdb tcp::26000	通过指定 TCP 端口告诉 QEMU 在该端口上开启 GDB 调试服务器

调试

```
$ CPUS=1 make qemu-gdb
sed "s/:1234/:26000/" < .gdbinit.tmpl-riscv > .gdbinit
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

命令选项	含义
-S	让虚拟机在第一条指令之前就停下来，等待调试器（GDB）的后继指令。
-gdb tcp::26000	通过指定 TCP 端口告诉 QEMU 在该端口上开启 GDB 调试服务器

```
$ gdb-multiarch -x ./gdbinit
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
.....
The target architecture is set to "riscv:rv64".
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x00000000000001000 in ?? ()
(gdb)
```

调试

```
$ CPUS=1 make qemu-gdb
sed "s/:1234/:26000/" < .gdbinit.tmpl-riscv > .gdbinit
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

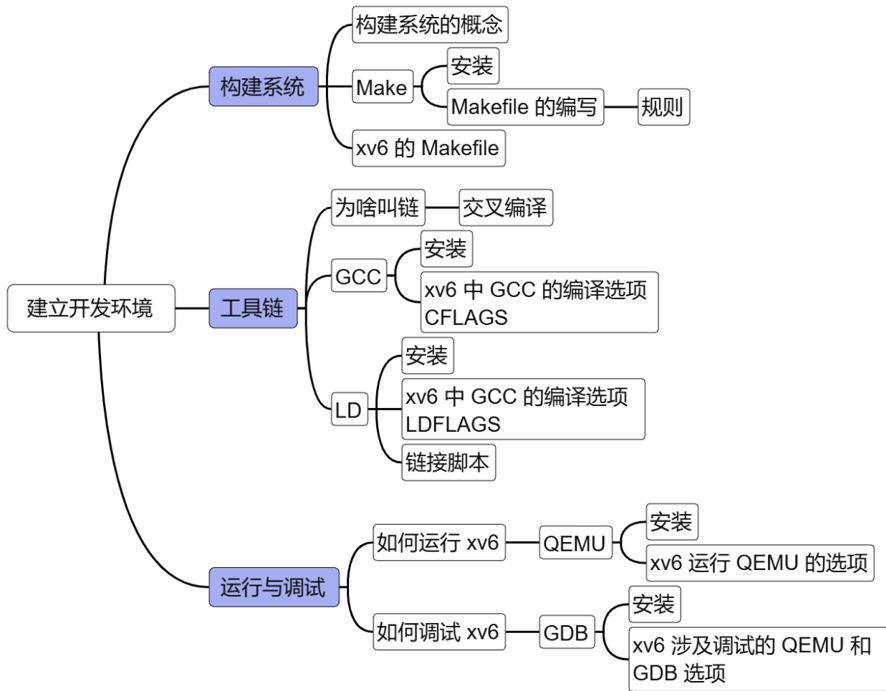
命令选项	含义
-S	让虚拟机在第一条指令处断点。
-gdb tcp::26000	通过指定 TCP 端口与 GDB 连接。

```
$ gdb-multiarch -x ./gdbinit
GNU gdb (Ubuntu 15.0.50.20240405) 15.0.50.20240405
Copyright (C) 2024 Free Software Foundation, Inc.
.....
The target architecture is set to "riscv:rv64".
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x00000000000001000 in ?? ()
(gdb)
```

```
set confirm off
set architecture riscv:rv64
target remote 127.0.0.1:26000
symbol-file kernel/kernel
set disassemble-next-line auto
set riscv use-compressed-breakpoints yes
```



本章总结



谢谢

